

MEEA* Code Experience

2022111931
이정진

Table of Contents

Code Recap

Train data

- provided
- non-provided
- GeLU

cpuct

- 0
- 15

policyNet

- provided
- transformer

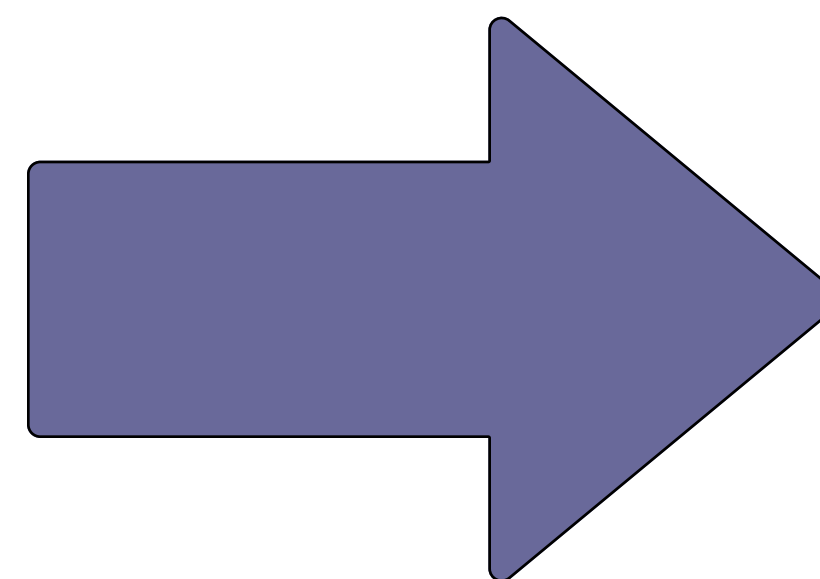
valueEnsemble

- provided
- 7

1. Code Recap

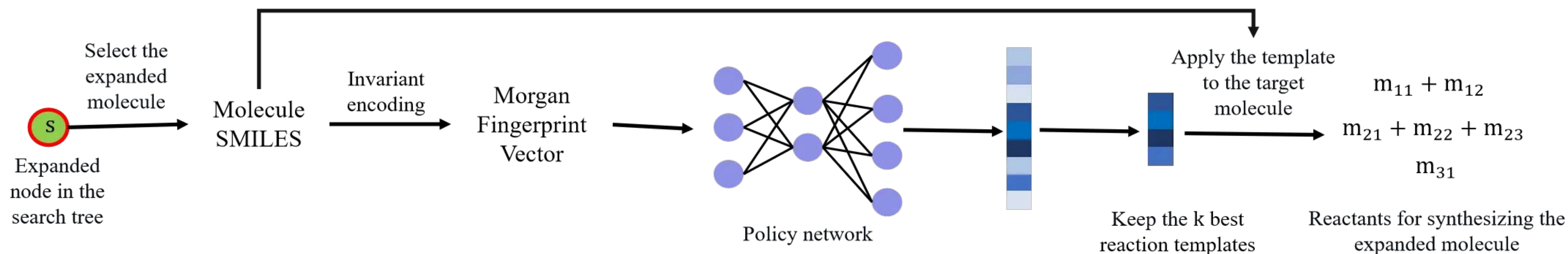
PolicyNet

- Single-Step retrosynthesis의 policy network
- 실제 전구체들의 집합을 찾는 역할
- Multi-Layer Perceptron (Dropout 0.1)



- 직접 학습을 진행
- Transformer로 변형
- 비선형 GELU 사용

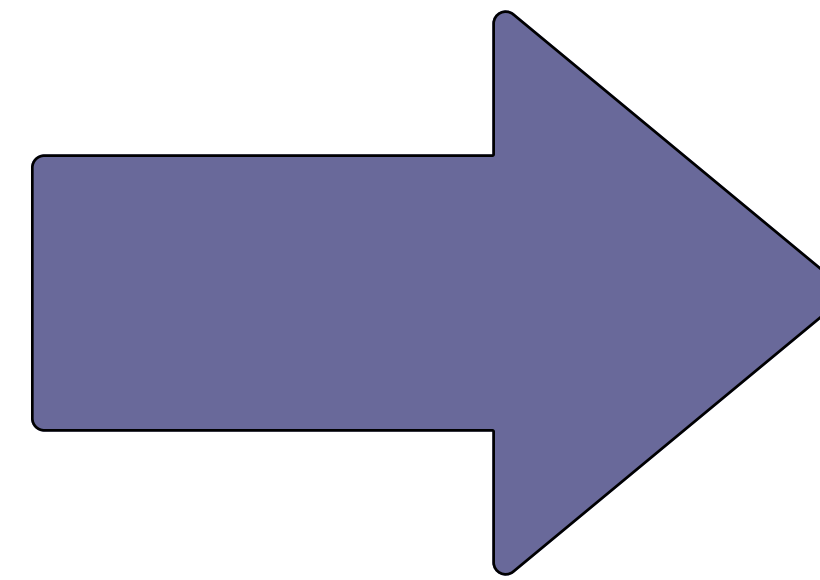
(c)



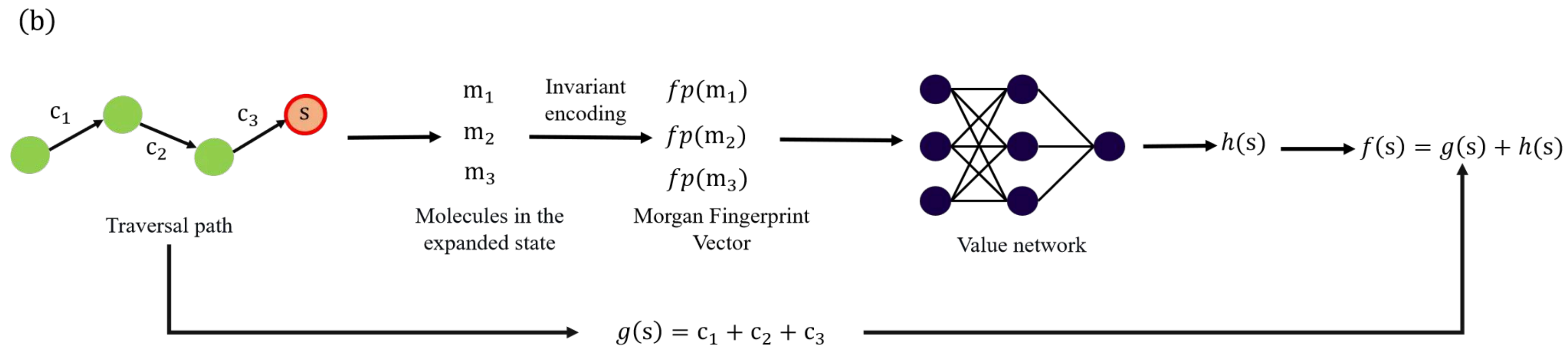
1. Code Recap

ValueEnsemble

- GraphModel로 Encoding한 뒤 linear layer를 통해 남은 경로(h^*)를 예측
- 예측값에 -7이라는 매직 넘버



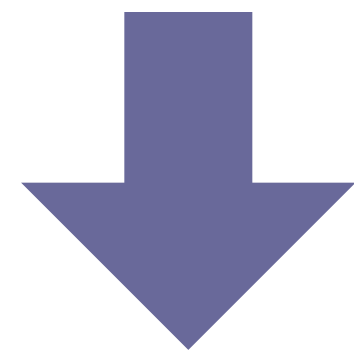
- 매직 넘버 삭제 후 학습
- 임의의 데이터로 학습



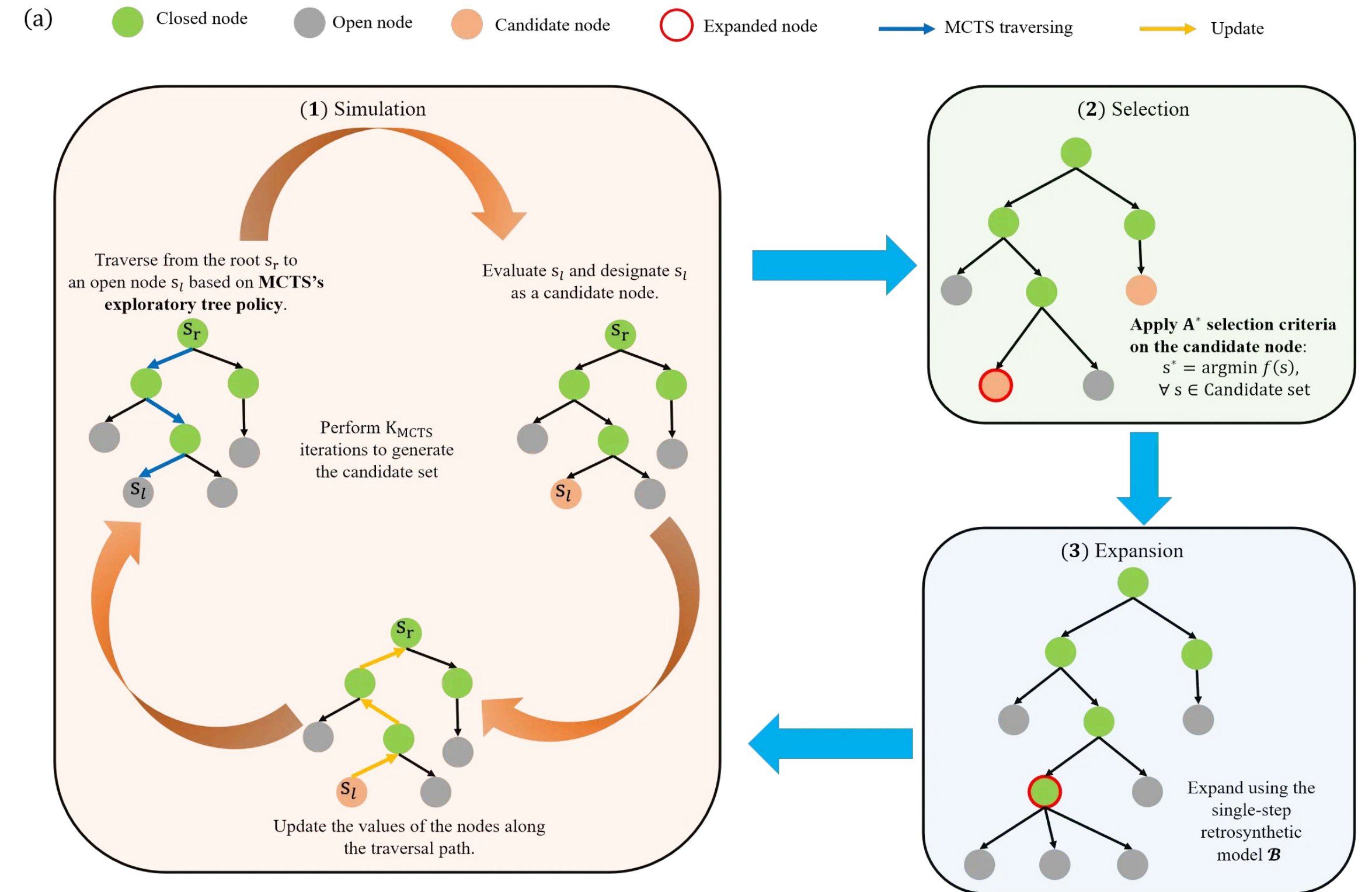
1. Code Recap

MEEA

- 실제 MEEA*-PC를 진행
- cpuct라는 hyperparameter를 통해 탐색 강도 조정
- cpuct가 0에 가까울수록 A*적인 성격
클수록 MCTS적인 성격



- cpuct 조정



2. Train data

Provided



policy_model.ckpt



template_rules.dat



value_pc.pt



building_blocks.txt



origin_dict.csv



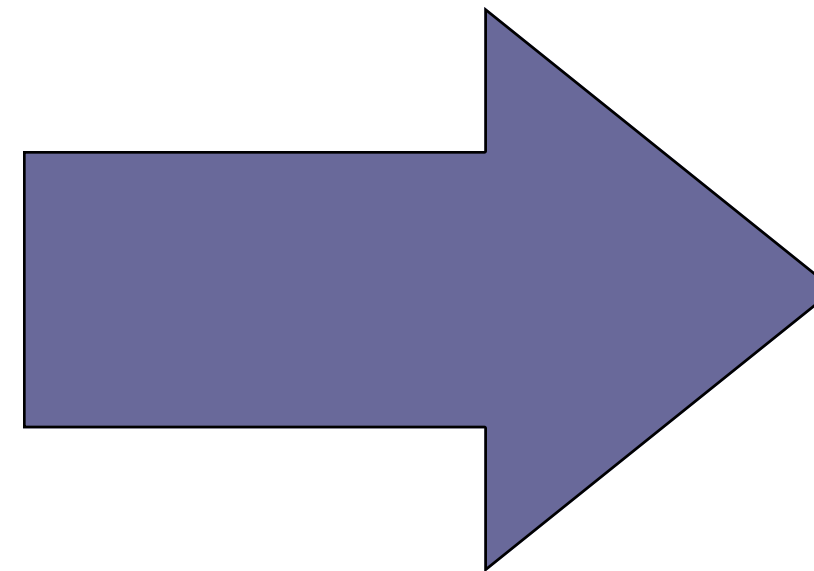
routes_possible_test_hard.pkl

- **policy_model.ckpt**
: 사전학습된 정책 네트워크
- **template_rules.dat**
: 역합성 반응 템플릿 라이브러리
- **value_pc.pt**
: 가치 함수 (Value Network) 모델
- **building_blocks.txt**
: 상업적으로 구매 가능한 시작 물질 목록
- **origin_dict.csv**
: 알려진 분자 데이터베이스
- **routes_possible_test_hard.pkl**
: 어려운 테스트 케이스 벤치마크

2. Train data

Non-Provided

- **uspto_template.pkl.gz**
: USPTO 템플릿 Dictionary
- **policyTrain.pkl.gz**
: policyNet 학습 데이터
- **policyTest.pkl.gz**
: policyNet 검증 데이터

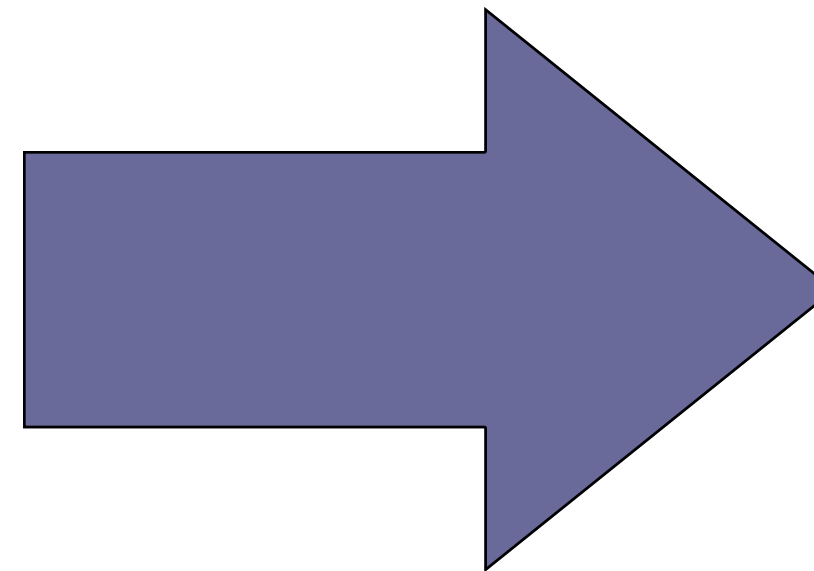


1. **USPTO 원본 데이터 확보 (TDC)**
2. **RXNMapper로 atom mapping**
3. **rdchiral로 template 새로 추출**
(label 개수 변화: 381,302 → 219,032)

2. Train data

Non-Provided

- **train_consistency.pkl**
: 경로 일관성 학습 데이터
- **train_fitting.pkl**
: 남은 경로 예측 학습 데이터
- **val_consistency.pkl**
: 경로 일관성 검증 데이터
- **val_fitting.pkl**
: 남은 경로 검증 데이터



- **학습 데이터 취득 방식 (개념적)**
USPTO(299k)를 MEEA* 수행한 뒤 모든 경로를 저장 (재현이 거의 불가능)

코드 제공 X
- **routes_possible_test_hard.pkl**
→ train/validate(9:1)

3. cpuct

0 (A*)

- 평균 Depth가 18로 깊은 편
- 80%의 정확도

```
[(base) earthenergy1128@gpusystem:~/MEEA$ cat result_simulation.txt  
USPTO      100      500      0.0      0.8000  18.63   3282.80 00:54:42
```

3. cpuct

15 (MCTS)

- 평균 Depth가 6으로 얇은 편
- 94%의 정확도
- USPTO가 비교적 낮은 depth를 필요로 하는 걸 고려했을 때 정상적인 결과

```
(base) earthenergy1128@gpusystem:~/MEEA$ cat result_simulation_MCTS.txt
USPTO    100      500      15.0      0.9421   6.91      916.91   00:15:16
```

4. policyNet

Provided

- MLP (fully connected 2개, dropout(0.1) 1개)

Training results

Top 1: 0.3405283773647804
Top 10: 0.5619079367725088
Top 50: 0.6186626663333056
validation loss ⇒ 0.004996587834700551

USPTO

precision: 0.8000
depth: 13.73
time(second): 1705.53
time(HH:MM:SS): 00:28:25

Natural

precision: 0.0053
depth: 31.87
time(second): 8425.09
time(HH:MM:SS): 02:20:25

4. policyNet

GELU

- Non Linear Function을 통한 표현력 증가 기대

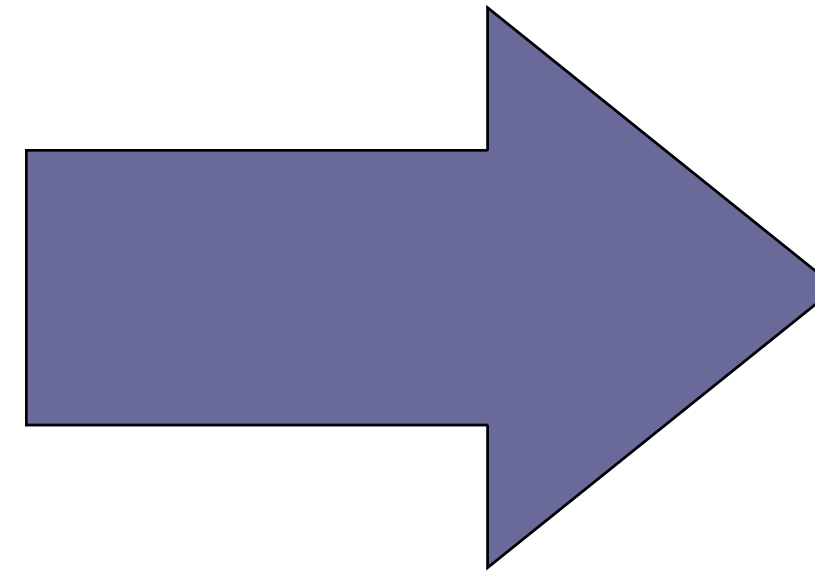
Training results

Top 1: 0.3412182496689539

Top 10: 0.5626255891694678

Top 50: 0.6187830467353761

validation loss $\Rightarrow$ 0.00497747625562244



미약하지만 성능의 개선은
있었음

4. policyNet

GELU

USPTO

precision: 0.9474

depth: 7.89

time(second): 736.91

time(HH:MM:SS): 00:12:16

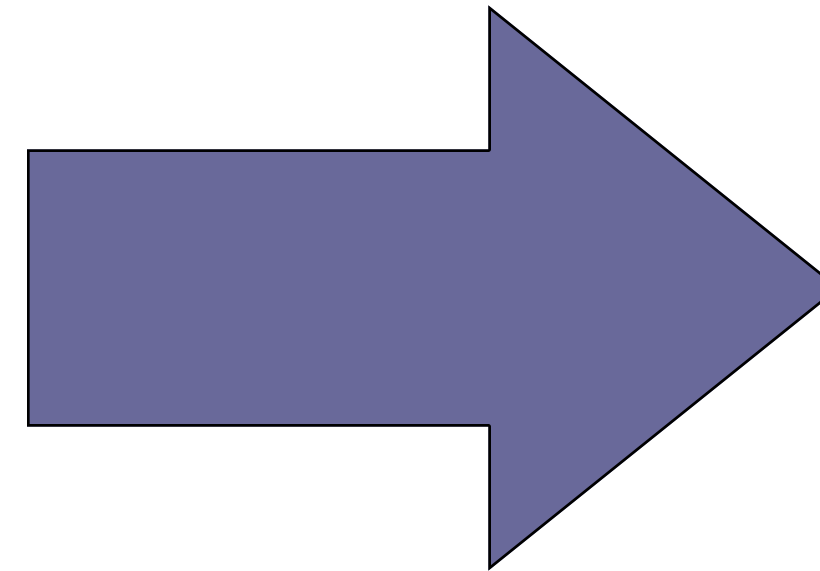
Natural

precision: 0.0027

depth: 31.92

time(second): 8417.40

time(HH:MM:SS): 02:20:17



USPTO의 성능은 크게 향상

자연물에서의 성능은 떨어진 걸로
나오지만 테스트 케이스가 적고(약
400개) 차이도 크지 않음 (유의미
한 결과 X)

4. policyNet

Transformer

```
class PositionalEncoding(nn.Module):
    """Sinusoidal positional encoding used by the Transformer encoder."""

    def __init__(self, d_model: int, max_len: int = 4096):
        super().__init__()
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2) * (-torch.log(torch.tensor(10000.0)) / d_model))
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0) # (1, max_len, d_model)
        self.register_buffer("pe", pe)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, seq_len, d_model)
        seq_len = x.size(1)
        return x + self.pe[:, :seq_len]
```

- fingerprint 벡터를 patch 단위로 분리 → Linear embedding (토큰화)
- 이후 순서에 따라 상대적 거리 부여
- 트랜스포머를 통해 국소적 거리에 따른 특징 추출 기대

4. policyNet

Transformer

Training results

Top 1: 0.26972618088544414

Top 10: 0.49040660795807056

Top 50: 0.5667416729171876

validation loss $\Rightarrow$ 0.012327133880108544

- MLP보다 떨어진 성능을 보임
- FingerPrint에서는 분자의 시퀀스에 따른 특징이 미약
- SMILES와 같이 위치 순서가 상관 있는 모델에 더 적합할 것으로 예상

4. policyNet

Transformer

USPTO

precision: 0.7000

depth: 15.94

time(second): 1288.67

time(HH:MM:SS): 00:21:28

Natural

precision: 0.0000

depth: 32

time(second): 8417.75

time(HH:MM:SS): 02:20:17

5. valueEnsemble

7 (magic number) vs without 7

Training results (loss)

Epoch 1029300

total: 6.2476

fit: 3.0891

cons: 3.1585

vs

Training results (loss)

Epoch 208200

total: 0.0236

fit: 0.0161

cons: 0.0075

7이 없을 때 loss가 작아서 좋아 보이지만 hinge loss에 의한 학습을 고려했을 때 gradient vanishing 우려가 타당해보임

6. 추가적인 연구 방향

PolicyNet 변형

- SMILES 데이터를 통해 Morgan Fingerprint보다 풍부한 표현으로 transformer와 같은 복잡한 모델로 분자 구조를 임베딩하여 정확한 정책 학습 가능
- 그대로 Morgan fingerprint 데이터를 사용하되 GNN을 통해 반응 중심의 예측

6. 추가적인 연구 방향

ValueEnsemble 변형

- Magic number 7을 사용하기보다 Adaptive Threshold를 사용
- Consistency Loss에 KL-divergence와 같은 분포 정규화 기법을 사용